Звіт

з лабораторної роботи 4

“Dining philosophers”

з навчальної дисципліни «Розподілені системи обробки інформації»

студента групи ММШІ-1

Красікова Антона Ігоровича

1. Постановка задачі

П’ятеро філософів сидять за круглим столом, маючи перед собою по одній тарілці спагеті. Між кожними двома тарілками лежить одна виделка (загалом 5 виделок). Для того щоб їсти, філософу необхідні дві виделки - ліва та права. Філософи можуть перебувати у двох основних станах: думати або їсти. Філософ не знає про наміри своїх сусідів і діє незалежно. Після прийому їжі він кладе обидві виделки назад на стіл.

Мета роботи: розробити конкурентний алгоритм, який гарантує відсутність взаємоблокувань (deadlock) та нескінченного голодування філософів. Необхідно реалізувати два варіанти розв'язку:

1. Для N = 5 з використанням одного лічильного семафора.
2. Для N = 5 з використанням м’ютекса та п’яти семафорів (алгоритм Таненбаума/Дейкстри).

2. Спосіб розв’язання

Оскільки базовий алгоритм (коли кожен філософ бере спочатку ліву виделку, а потім праву) неминуче призводить до взаємоблокування у випадку одночасного «зголодніння» всіх філософів, було застосовано два класичні архітектурні підходи:

- **Підхід 1: Обмеження доступу (Лічильний семафор / "Офіціант")** Алгоритм не змінює порядок взяття виделок, але вводить штучне обмеження: одночасно намагатися взяти виделки можуть не більше ніж N-1 (тобто 4) філософів. Для цього використовується лічильний семафор, ініціалізований значенням 4. Оскільки за столом 5 виделок і максимум 4 претенденти, як мінімум один філософ завжди знайде обидві виделки вільними. Це робить виникнення deadlock математично неможливим.
- **Підхід 2: Алгоритм Таненбаума (М'ютекс + масив семафорів)** У цьому підході виделки не є окремими ресурсами (м'ютексами), які захоплюються по черзі. Замість цього вводиться поняття "стану" філософа (THINKING, HUNGRY, EATING). Глобальний м'ютекс використовується виключно для безпечної (атомарної) зміни цих станів. Філософ переходить у стан EATING тільки тоді, коли він голодний, а обидва його сусіди не їдять. Якщо сусіди зайняті, філософ блокується на своєму персональному бінарному семафорі. Коли сусід закінчує їсти, він перевіряє стан своїх сусідів і "будить" їх через семафор, якщо вони чекали.

3. Деталі реалізації

Програму розроблено мовою **C++ (стандарт C++20)** з дотриманням наступних практик:

- **Патерн "Стратегія" (Strategy):** Логіку синхронізації винесено в окремі класи, що реалізують інтерфейс IDiningStrategy. Це дозволило уникнути дублювання коду самого циклу життя філософа та легко перемикатися між алгоритмами через окремий флаг.
- **Сучасні примітиви синхронізації:** Було використано бібліотеку <semaphore> (std::counting_semaphore та std::binary_semaphore).
- **Безпечне керування потоками:** Використано std::jthread, який гарантує автоматичний join() при знищенні об'єкта, запобігаючи аварійному завершенню програми, характерному для від'єднаних потоків.
- **Уникнення Data Race:** Генератор випадкових чисел для таймерів (std::default_random_engine) визначено як thread_local. Це гарантує, що кожен потік має власну ізольовану копію генератора, усуваючи стан гонки при конкурентному доступі.
- **Керування пам'яттю:** Використано смарт-вказівники (std::unique_ptr) та стандартні контейнери (std::array, std::vector), що повністю виключає витоки пам'яті, які могли б виникнути при використанні сирих вказівників.

4. Результати та код

Фрагмент роботи алгоритму з лічильним семафором:

```
T W T E T  Total eaters: 1
T E T E T  Total eaters: 2
W E T E T  Total eaters: 2
W E T S T  Total eaters: 1
W E T T T  Total eaters: 1
W E T T W  Total eaters: 1
W E W T W  Total eaters: 1
W E W W W  Total eaters: 1
W S W W W  Total eaters: 0
W S E W W  Total eaters: 1
W T E W W  Total eaters: 1
E T E W W  Total eaters: 2
E W E W W  Total eaters: 2
E W S W W  Total eaters: 1
E W T W W  Total eaters: 1
S W T W W  Total eaters: 0
T W T W W  Total eaters: 0
T E T W W  Total eaters: 1
T E T W E  Total eaters: 2
T S T W E  Total eaters: 1
T T T W E  Total eaters: 1
```

Фрагмент роботи алгоритму Таненбаума:

```
E T W E T  Total eaters: 2
E T W E W  Total eaters: 2
E W W E W  Total eaters: 2
E W W S W  Total eaters: 1
E W W T W  Total eaters: 1
E W E T W  Total eaters: 2
S W E T W  Total eaters: 1
T W E T W  Total eaters: 1
T W E T E  Total eaters: 2
T W E T S  Total eaters: 1
T W E T T  Total eaters: 1
W W E T T  Total eaters: 1
E W E T T  Total eaters: 2
E W S T T  Total eaters: 1
E W T T T  Total eaters: 1
E W T W T  Total eaters: 1
E W T E T  Total eaters: 2
S W T E T  Total eaters: 1
T W T E T  Total eaters: 1
T E T E T  Total eaters: 2
T E T S T  Total eaters: 1
T E T T T  Total eaters: 1
T E T T W  Total eaters: 1
T E T T E  Total eaters: 2
W E T T E  Total eaters: 2
```

Висновок: Обидва реалізовані алгоритми успішно працюють без виникнення ситуацій взаємоблокування. Під час тестування спостерігалося, що зазвичай одночасно можуть їсти від 1 до 2 філософів. Короткочасні моменти, коли жоден філософ не їсть (0 eaters), є нормальною поведінкою конкурентної системи, що пояснюється затримками на перемикання контексту потоків та випадковими фазами "роздумів", у які потоки переходять незалежно один від одного.

Посилання на код (папка lab4):

`https://github.com/chesswondo/distributed-systems-labs`